

PROBLEM DECOMPOSITION

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain **problem decomposition** and **stepwise refinement**.
 - Define and use **functions** and **procedures** in pseudocode.
 - Distinguish between **parameters** and **variable scope**.
 - Apply modular design to develop a simple **calculator algorithm**.
-

1. Introduction to Problem Decomposition

Complex problems can be difficult to solve all at once.

Problem decomposition means **breaking a large problem into smaller, more manageable parts (sub-problems)**.

Each sub-problem can then be solved independently and combined to form the complete solution.

2. Stepwise Refinement

Stepwise refinement is the process of developing an algorithm in **successive steps**, refining each level of detail.

Example – Compute and Display the Average of 3 Numbers

Step 1: State the problem.

Find the average of three numbers.

Step 2: Break it into sub-steps.

1. Get three numbers.
2. Calculate their sum.
3. Divide the sum by 3.
4. Display the average.

Step 3: Express as pseudocode.

```
INPUT A, B, C
sum = A + B + C
average = sum / 3
PRINT "Average = ", average
```



3. Functions and Procedures in Pseudocode

After decomposition, we often reuse parts of an algorithm.
To do this efficiently, we define **functions** or **procedures**.

Type	Returns a Value?	Typical Use
Function	Yes	Calculate and return a result
Procedure	No	Perform an action without returning a value

⚙️ 3.1 Function Syntax

```
FUNCTION FunctionName(parameters)
    statements
    RETURN value
ENDFUNCTION
```

Example:

```
FUNCTION Add(x, y)
    sum = x + y
    RETURN sum
ENDFUNCTION
```

3.2 Procedure Syntax

```
PROCEDURE ProcedureName(parameters)
    statements
ENDPROCEDURE
```

Example:

```
PROCEDURE DisplayResult(result)
    PRINT "The result is ", result
ENDPROCEDURE
```

3.3 Calling Functions and Procedures

```
sum = Add(5, 10)
CALL DisplayResult(sum)
```

4. Parameters and Variable Scope

Parameters

Values passed **into** a function or procedure.

- **By Value:** a copy is passed – original unchanged.
- **By Reference:** a reference is passed – original can change.

Example:

```
PROCEDURE Increase(num BYREF)
    num = num + 1
ENDPROCEDURE
```

Variable Scope

- **Local Variables:** declared inside a function or procedure, accessible only there.
- **Global Variables:** declared outside and accessible throughout the program.

Example:

globalCount = 0

```
PROCEDURE IncrementCount()
    localCount = 1
    globalCount = globalCount + 1
    PRINT "Local:", localCount, "Global:", globalCount
ENDPROCEDURE
```

5. Activity – Design a Modular Calculator Algorithm

Problem

Design a simple calculator that can add, subtract, multiply, and divide two numbers using modular (pseudocode) functions.

Step 1: Decompose the Problem

1. Input two numbers and an operator.
 2. Define separate functions for each operation.
 3. Use a selection structure (CASE or IF) to call the correct function.
 4. Display the result.
-

Step 2: Pseudocode

PROCEDURE Main()

 INPUT num1, num2, operator

 CASE operator OF

 "+": result = Add(num1, num2)

 "-": result = Subtract(num1, num2)

 "*": result = Multiply(num1, num2)

 "/": result = Divide(num1, num2)

 OTHERWISE: PRINT "Invalid operator"

 ENDCASE

 PRINT "Result = ", result

ENDPROCEDURE

FUNCTION Add(a, b)

 RETURN a + b

ENDFUNCTION

FUNCTION Subtract(a, b)

 RETURN a - b

ENDFUNCTION

FUNCTION Multiply(a, b)

 RETURN a * b

ENDFUNCTION

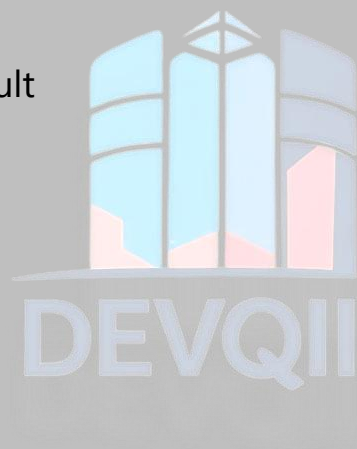
FUNCTION Divide(a, b)

 IF b = 0 THEN

 PRINT "Error: Division by zero"

 RETURN 0

 ELSE



```
    RETURN a / b
ENDIF
ENDFUNCTION
```

Step 3: Flowchart Concept

A flowchart could include:

- Input block for numbers and operator.
 - Decision diamond to choose operation.
 - Process boxes for Add/Subtract/Multiply/Divide.
 - Output block for result.
-

6. Exercises

1. Write a function that returns the **square** of a number.
 2. Write a procedure that **prints the average** of two numbers.
 3. Modify the calculator to include a **modulus (%)** operation.
 4. Explain the difference between local and global variables using examples.
-